

Reviewer Pack — Sandpile-Group 2-Rank Lower-Bounds Tseitin DPLL Tree Size

Entry d46d829f1f79 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:55:02 UTC

Sandpile-Group 2-Rank Lower-Bounds Tseitin DPLL Tree Size

Entry ID: d46d829f1f79 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 06:55:02 UTC

1. Conjecture

Field A (mathematical branch): Chip-firing / abelian sandpile combinatorics: the critical (sandpile) group $K(G)$ of a connected graph, recovered as $\mathbb{Z}^{V-1}/\text{im}(L'(G))$ from the reduced graph Laplacian $L'(G)$, with its invariant-factor decomposition obtained by Smith normal form over $\mathbb{Z}$; the 2-rank $k_2(G) := \dim_{\mathbb{F}_2}(K(G)/2K(G))$ is a polynomial-time graph invariant essentially absent from the proof-complexity literature **Field B** (complexity object): Resolution / DPLL refutation length of Tseitin formulas $\text{Tseitin}(G, \sigma)$ on 3-regular graphs (the Urquhart-style canonical sub-Frege gateway target whose Frege proof complexity remains open and is the natural launchpad for any EF lower-bound program in the spirit of Krajicek-Buss-Razborov-Impagliazzo)

Statement:

For every connected 3-regular graph G on V vertices and every charge $\sigma \in \mathbb{F}_2^V$ with parity sum 1, let $k_2(G) := \dim_{\mathbb{F}_2}(K(G)/2K(G))$ be the 2-rank of the sandpile group (equivalently $V-1\text{-rank}_{\mathbb{F}_2}(L' \bmod 2)$) and let $T(G, \sigma)$ be the number of decision nodes in the DPLL search tree for $\text{Tseitin}(G, \sigma)$ using fixed-lex variable order with unit propagation. Then $\log_2 T(G, \sigma) \geq k_2(G)$; a single (G, σ) with $T(G, \sigma) < 2^{k_2(G)}$ refutes the conjecture.

Rationale (proposer's reasoning):

$\text{Tseitin}(G, \sigma)$ encodes solvability of an $\mathbb{F}_2$ -linear system whose obstruction is precisely the image of σ under the $\mathbb{F}_2$ -augmentation of the integral cycle/cut sequence $\mathbb{Z}^E \rightarrow \mathbb{Z}^V \rightarrow K(G)$; the 2-rank $k_2(G)$ measures how much of that obstruction survives integrally, so a refutation must implicitly certify each independent $\mathbb{F}_2$ -cocycle generator in $K(G)$. This connects an integral-lattice invariant of expanders to proof length without invoking algebraic geometry or representation theory. The bridge from chip-firing combinatorics to Frege/EF proof-complexity targets is, to our knowledge, untouched, and gives a falsifiable shadow on the resolution sub-system whose extension to bounded-depth Frege depth is the natural follow-up.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 54be4654cf68400e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 instances (5 sizes $V \in \{8, 10, 12, 14, 16\} \times 30$ seeds), with $k_2(G)$ from integer SNF of reduced Laplacian and $T(G, \sigma)$ decision-node count from fixed-lex DPLL+UP+pure-literal, the conjecture is SUPPORTED iff every instance satisfies $\log_2 T(G, \sigma) \geq k_2(G)$; any single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.97	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formulas resolution lower bound sandpile group Laplacian - DPLL Tseitin 3-regular graph F₂ rank Laplacian expansion - critical group 2-rank Smith normal form proof complexity Tseitin

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def is_connected(G):
    visited = set()
    stack = [0]
    while stack:
        u = stack.pop()
        if u not in visited:
            visited.add(u)
            for v in G[u]:
                if v not in visited:
                    stack.append(v)
    return len(visited) == len(G)

def laplacian_matrix(G):
    n = len(G)
    L = [[0] * n for _ in range(n)]
    degree_sum = sum(len(neighbors) for neighbors in G.values())
```

```

for u, neighbors in enumerate(G):
    L[u][u] = -len(neighbors)
    for v in neighbors:
        L[u][v] += 1
return L

def smith_normal_form(M):
    n = len(M)
    U, V = [[0] * n for _ in range(n)], [[0] * n for _ in range(n)]
    for i in range(n):
        U[i][i], V[i][i] = 1, 1
    for k in range(n):
        # Find pivot
        pivot_row, pivot_col = -1, -1
        for i in range(k, n):
            for j in range(k, n):
                if M[i][j] != 0:
                    pivot_row, pivot_col = i, j
                    break
            if pivot_row != -1:
                break

        # Swap rows to make the pivot non-zero
        if pivot_row != k:
            M[k], M[pivot_row] = M[pivot_row], M[k]
            U[k], U[pivot_row] = U[pivot_row], U[k]

        # Make the pivot 1
        factor = M[k][k]
        for j in range(n):
            M[k][j] //= factor
            U[k][j] //= factor

        # Eliminate below and above the pivot
        for i in range(k + 1, n):
            factor = M[i][k]
            for j in range(n):
                M[i][j] -= factor * M[k][j]
                U[i][j] -= factor * U[k][j]

        for i in range(k - 1, -1, -1):
            factor = M[i][k]
            for j in range(n):
                M[i][j] -= factor * M[k][j]
                U[i][j] -= factor * U[k][j]

    return M, U, V

def dpll(G, assignment, unit_clause=None):
    if unit_clause is not None:
        u, v = unit_clause
        if v in G[u]:
            assignment[v] = 1
        else:
            assignment[v] = 0

```

```

# Unit propagation
while True:
    changed = False
    for v in range(len(G)):
        if assignment[v] is not None:
            continue
        ones, zeros = 0, 0
        for u in G[v]:
            if assignment[u] == 1:
                ones += 1
            elif assignment[u] == 0:
                zeros += 1
        if ones == len(G[v]):
            assignment[v] = 0
            changed = True
        elif zeros == len(G[v]) - 1:
            assignment[v] = 1
            changed = True
    if not changed:
        break

# Pure literal elimination
while True:
    changed = False
    for u in range(len(G)):
        ones, zeros = 0, 0
        for v in G[u]:
            if assignment[v] == 1:
                ones += 1
            elif assignment[v] == 0:
                zeros += 1
        if ones == len(G[u]):
            for v in G[u]:
                if assignment[v] is None:
                    assignment[v] = 0
                    changed = True
        elif zeros == len(G[u]) - 1:
            for v in G[u]:
                if assignment[v] is None:
                    assignment[v] = 1
                    changed = True
    if not changed:
        break

# Backtracking
stack = [(0, {})]
while stack:
    u, assignment = stack.pop()
    if u == len(G):
        return True

    for v in G[u]:
        if assignment[v] is None:
            new_assignment = assignment.copy()
            new_assignment[v] = 1
            if dp11(G, new_assignment, (u, v)):

```

```

        return True
        new_assignment[v] = 0
        stack.append((u + 1, new_assignment))

    return False

def tseitin_encoding(G, sigma):
    n = len(G)
    clauses = []
    for u in range(n):
        if sigma[u]:
            clauses.append([u])
        else:
            clauses.append([-u])

    for u in range(n):
        for v in G[u]:
            clauses.append([u, -v])
            clauses.append([-u, v])

    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    V = random.choice([8, 10, 12, 14, 16])
    G = {}
    while not is_connected(G):
        G.clear()
        for _ in range(V * (V - 1) // 2):
            u, v = random.sample(range(V), 2)
            if u != v and v not in G[u]:
                G.setdefault(u, []).append(v)
                G.setdefault(v, []).append(u)

    sigma = [i == 0 for i in range(V)]
    L = laplacian_matrix(G)
    M = [[L[i][j] % 2 for j in range(len(L))] for i in range(len(L))]
    k_2_G, _, _ = smith_normal_form(M)
    k_2_G = sum(1 for row in k_2_G if any(x != 0 for x in row))

    clauses = tseitin_encoding(G, sigma)
    assignment = [None] * V
    decision_nodes = 0

    def dpll_with_count(G, assignment, unit_clause=None):
        nonlocal decision_nodes
        if unit_clause is not None:
            u, v = unit_clause
            if v in G[u]:
                assignment[v] = 1
            else:
                assignment[v] = 0

        while True:
            changed = False
            for v in range(len(G)):

```

```

        if assignment[v] is not None:
            continue
        ones, zeros = 0, 0
        for u in G[v]:
            if assignment[u] == 1:
                ones += 1
            elif assignment[u] == 0:
                zeros += 1
        if ones == len(G[v]):
            assignment[v] = 0
            changed = True
        elif zeros == len(G[v]) - 1:
            assignment[v] = 1
            changed = True
    if not changed:
        break

while True:
    changed = False
    for u in range(len(G)):
        ones, zeros = 0, 0
        for v in G[u]:
            if assignment[v] == 1:
                ones += 1
            elif assignment[v] == 0:
                zeros += 1
        if ones == len(G[v]):
            for v in G[u]:
                if assignment[v] is None:
                    assignment[v] = 0
                    changed = True
        elif zeros == len(G[v]) - 1:
            for v in G[u]:
                if assignment[v] is None:
                    assignment[v] = 1
                    changed = True
    if not changed:
        break

stack = [(0, {})]
while stack:
    u, assignment = stack.pop()
    decision_nodes += 1
    if u == len(G):
        return True

    for v in G[u]:
        if assignment[v] is None:
            new_assignment = assignment.copy()
            new_assignment[v] = 1
            if dpll_with_count(G, new_assignment, (u, v)):
                return True
            new_assignment[v] = 0
            stack.append((u + 1, new_assignment))

return False

```

```

dpll_with_count(G, assignment)

log_2_T_G_sigma = math.log2(decision_nodes) if decision_nodes > 0 else -math.inf

return {
    "metric_name": "log_2 T(G, $\sigma$ )",
    "metric_value": log_2_T_G_sigma,
    "instances_tested": 1,
    "conjecture_holds": log_2_T_G_sigma >= k_2_G,
    "counterexample": "" if log_2_T_G_sigma >= k_2_G else f"V={V}, k_2(G)={k_2_G}, T(G, $\sigma$ )={dec
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899352df.py", line 276, in <module>
    result = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899352df.py", line 171, in run_trial
    while not is_connected(G):
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_899352df.py", line 24, in is_connected
    for v in G[u]:
    ~~~

```

KeyError: 0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `KeyError` in `is_connected` before any instances were evaluated, so no data on $\log_2 T(G, \sigma)$ vs $k_2(G)$ was produced. Per Rule 2, a crash yields INCONCLUSIVE. | next: Fix the graph construction/representation bug (ensure `G` is a dict keyed by all `V` vertices before calling `is_connected`) and rerun the 150-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	181614
2	preregistration	claude_max	opus	0	0	6481
3	novelty	claude_max	opus	0	0	5543
4	novelty	claude_max	opus	0	0	8992
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	72290
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49205
7	judge	claude_max	opus	0	0	5277

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 329401 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/d46d829f1f79.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d46d829f1f79.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d46d829f1f79.tar.gz` (if generated)